

Практическое занятие 3. Добавляем собственный веб-интерфейс

Цель: На уже готовом бэкенде (модель, валидация, админка, middleware, настройки) создать свой веб-интерфейс для работы с товарами: список, добавление, редактирование, удаление. Это даст дополнительные 20 баллов на экзамене.

Время выполнения: около 40 минут, если бэкенд уже готов.

Предполагается, что у вас есть:

- Проект `exam_project`
 - Приложение `products`
 - Модель `Product` (поля: `name`, `category`, `price`, `sku`, `created_at`)
 - Админ-панель работает, валидация есть, `middleware`, `.env`, статика, тест — всё настроено.
-

Шаг 1. Создаём папку для шаблонов

В вашем приложении `products` создайте папку `templates`, а внутри неё – папку `products` (полный путь: `products/templates/products/`).

Шаг 2. Создаём общий базовый шаблон `base.html`

В папке `products/templates/products/` создайте файл `base.html`.
Скопируйте в него:

```
html
```

```
<!DOCTYPE html>

<html lang="ru">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>{% block title %}Управление товарами{% endblock %}</title>

  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.c
ss" rel="stylesheet">

</head>

<body>

  <div class="container mt-4">

    {% block content %}{% endblock %}

  </div>

</body>

</html>
```

Это каркас, который будет использоваться всеми страницами. Он подключает Bootstrap и определяет два «блока» — `title` и `content`, которые будут заполняться в дочерних шаблонах.

Шаг 3. Создаём шаблон списка товаров

`product_list.html`

В той же папке создайте файл `product_list.html`:

```
html
```

```
{% extends 'products/base.html' %}

{% block title %}Список товаров{% endblock %}

{% block content %}

<h1>Товары</h1>

<a href="{% url 'product_create' %}" class="btn btn-primary mb-3">Добавить
товар</a>

<table class="table table-striped">

    <thead>

        <tr>

            <th>ID</th>

            <th>Название</th>

            <th>Категория</th>

            <th>Цена</th>

            <th>Артикул</th>

            <th>Действия</th>

        </tr>

    </thead>

    <tbody>

        {% for product in products %}

            <tr>

                <td>{{ product.id }}</td>

                <td>{{ product.name }}</td>

                <td>{{ product.category }}</td>

                <td>{{ product.price }}</td>

                <td>{{ product.sku }}</td>

                <td>
```

```

        <a href="{% url 'product_update' product.id %}" class="btn
btn-sm btn-warning">Изменить</a>

        <a href="{% url 'product_delete' product.id %}" class="btn
btn-sm btn-danger">Удалить</a>

    </td>

</tr>

{% empty %}

<tr><td colspan="6">Нет товаров. Добавьте первый.</td></tr>

{% endfor %}

</tbody>

```

七八

```
{% endblock %}
```

Обратите внимание:

- `{% url 'product_create' %}` и другие — это имена маршрутов, которые мы создадим позже. Пока они не работают, но мы их пропишем.
- В цикле `for product in products` — переменная `products` будет передана из представления.

Шаг 4. Создаём шаблон формы `product_form.html`

Файл `product_form.html`:

```

html

{% extends 'products/base.html' %}

{% block title %}{% if product %}Редактирование{% else %}Создание{% endif
%}{% endblock %}

```

```
{% block content %}

<h1>{% if product %}Редактирование товара{% else %}Новый товар{% endif
%}</h1>

<form method="post">

    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit" class="btn btn-success">Сохранить</button>

    <a href="{% url 'product_list' %}" class="btn btn-secondary">Отмена</a>

</form>

{% endblock %}
```

Этот шаблон будет использоваться и для создания, и для редактирования. Переменная `product` определяет, в каком режиме мы находимся (если `product` есть — редактирование, иначе создание). Форма `form` будет передана из представления.

Шаг 5. Создаём шаблон подтверждения удаления `product_confirm_delete.html`

```
html

{% extends 'products/base.html' %}

{% block title %}Удаление товара{% endblock %}

{% block content %}

<h1>Удалить товар?</h1>

<p>Вы уверены, что хотите удалить <strong>{{ product }}</strong>?</p>

<form method="post">
```

```
{% csrf_token %}

<button type="submit" class="btn btn-danger">Да, удалить</button>

<a href="{% url 'product_list' %}" class="btn btn-secondary">Нет,
отмена</a>

</form>

{% endblock %}
```

Шаг 6. Создаём форму (forms.py)

Форма нужна для отображения полей на HTML-странице и для валидации данных. Создайте файл `products/forms.py`:

```
python

from django import forms

from .models import Product

class ProductForm(forms.ModelForm):

    class Meta:

        model = Product

        fields = ['name', 'category', 'price', 'sku']

        widgets = {

            'name': forms.TextInput(attrs={'class': 'form-control'}),

            'category': forms.TextInput(attrs={'class': 'form-control'}),

            'price': forms.NumberInput(attrs={'class': 'form-control',
'step': '0.01'}),

            'sku': forms.TextInput(attrs={'class': 'form-control'}),

        }
```

Пояснение:

- `ModelForm` – автоматически создаёт поля на основе модели `Product`.
- `fields` – перечисляем, какие поля модели должны быть в форме (поля `id` и `created_at` не нужны, они заполняются автоматически).
- `widgets` – задаём HTML-атрибуты для полей: добавляем классы Bootstrap (`form-control`) и для `price` – шаг 0.01 для удобного ввода копеек.

333

Шаг 7. Создаём представления (views.py)

Откройте `products/views.py`. Если там уже есть какие-то старые представления от предыдущих занятий, вы можете добавить новые. Мы создадим четыре функции: для списка, создания, редактирования и удаления.

Полный код `views.py` для собственного интерфейса:

```
python

from django.shortcuts import render, redirect, get_object_or_404
from .models import Product
from .forms import ProductForm

def product_list(request):
    """Список всех товаров, отсортированных по дате (сначала новые)"""
    products = Product.objects.all().order_by('-created_at')

    return render(request, 'products/product_list.html', {'products':
products})

def product_create(request):
    """Создание нового товара"""
```

```
if request.method == 'POST':

    form = ProductForm(request.POST)

    if form.is_valid():

        form.save()

        return redirect('product_list') # после сохранения – на список

else:

    form = ProductForm()

    return render(request, 'products/product_form.html', {'form': form,
'product': None})

def product_update(request, pk):

    """Редактирование товара по его id (pk)"""

    product = get_object_or_404(Product, pk=pk)

    if request.method == 'POST':

        form = ProductForm(request.POST, instance=product)

        if form.is_valid():

            form.save()

            return redirect('product_list')

    else:

        form = ProductForm(instance=product)

    return render(request, 'products/product_form.html', {'form': form,
'product': product})

def product_delete(request, pk):

    """Удаление товара с подтверждением"""

    product = get_object_or_404(Product, pk=pk)

    if request.method == 'POST':

        product.delete()
```



```
return redirect('product_list')

return render(request,
'products/product_confirm_delete.html', {'product': product})
```

Что здесь важно:

- `get_object_or_404` – если товар не найден, будет ошибка 404.
 - `instance=product` – при редактировании передаём существующий объект в форму, чтобы поля заполнились.
 - После успешного сохранения или удаления – перенаправление на страницу списка.
-

Шаг 8. Создаём URL-маршруты для приложения

Сначала нужно создать файл `products/urls.py` (если его нет). В нём пропишем маршруты, которые будут вести на наши представления.

```
python
```

```
from django.urls import path

from . import views

urlpatterns = [

    path('', views.product_list, name='product_list'),

    path('create/', views.product_create, name='product_create'),

    path('<int:pk>/update/', views.product_update, name='product_update'),

    path('<int:pk>/delete/', views.product_delete, name='product_delete'),

]
```

Пояснение:

- '' (пустая строка) – адрес /products/ (если потом подключим на префикс products/).
- 'create/' – адрес /products/create/.
- <int:pk> – это число (первичный ключ), например /products/5/update/.

Теперь нужно подключить эти маршруты в главном файле `exam_project/urls.py`. Откройте его и добавьте:

```
python

from django.contrib import admin

from django.urls import path, include

urlpatterns = [

    path('admin/', admin.site.urls),

    path('products/', include('products.urls')), # все адреса,
    начинающиеся с products/, идут в products.urls

]
```

Теперь ваш интерфейс будет доступен по адресу `http://127.0.0.1:8000/products/`.

Шаг 9. Проверка работы

1. Убедитесь, что сервер запущен: `python manage.py runserver`
2. Перейдите в браузере: `http://127.0.0.1:8000/products/`
3. Вы должны увидеть пустую таблицу (или уже добавленные через админку товары).
4. Попробуйте добавить товар через кнопку «Добавить товар». Заполните поля и нажмите «Сохранить».
5. Товар должен появиться в списке.
6. Проверьте редактирование (кнопка «Изменить») и удаление (кнопка «Удалить»).

Если что-то не работает:

- Проверьте, нет ли опечаток в именах URL (особенно в шаблонах: `{% url 'product_list' %}`).
 - Убедитесь, что в `products/urls.py` имена маршрутов совпадают с теми, что в шаблонах.
 - Убедитесь, что в `settings.py` приложение `products` добавлено в `INSTALLED_APPS`.
 - Проверьте, что форма `ProductForm` существует и импортирована в `views.py`.
-

Что делать, если вы хотите адаптировать этот интерфейс под другую модель (например, Event)?

1. Скопируйте все шаблоны и переименуйте их (например, `event_list.html`).
 2. Замените внутри шаблонов:
 - `product_list` → `event_list` (имена маршрутов).
 - `product` → `event` (имена переменных).
 - Названия полей в таблице (вместо `product.name` → `event.title` и т.д.).
 3. В `views.py` создайте аналогичные представления, заменив `Product` на `Event`.
 4. В `urls.py` добавьте маршруты для новой модели.
-

Критерии получения дополнительных 20 баллов на экзамене

- Веб-интерфейс работает без ошибок (список, создание, редактирование, удаление)
- Интерфейс адаптирован под вашу конкретную модель (поля названия, правильные ссылки) Итого: 20 баллов (добавляются к 80 баллам за бэкенд).

Заключение

Теперь у вас есть полноценный собственный интерфейс к вашей модели. Вы можете использовать этот шаблон для любой модели, меняя только названия полей и маршрутов. На экзамене вы сможете быстро добавить эту часть за 20-30 минут после выполнения основной бэкенд-части. Успешной подготовки!